

Problem Description: Task Scheduler with Dependencies

1. Overview

A Python module that computes a valid execution order for tasks with dependencies. Each task is identified by a unique string. A dependency (A, B) means A must complete before B begins. The module returns an ordered list of tasks such that every task appears after all of its dependencies.

1.1 API

Error Type	Condition
ValueError	Duplicate task names
ValueError	Dependency refers to unknown task
ValueError	Circular dependency exists
TypeError	tasks is not a list
TypeError	dependencies is not a list
TypeError	Task name is not a string

```
schedule_tasks(tasks: list[str], dependencies: list[tuple[str, str]]) -> list[str]
```

Parameter	Type	Required	Description
tasks	list[str]	Yes	List of unique task names
dependencies	list[tuple[str, str]]	Yes	Dependency pairs (before_task, after_task)

Returns: list[str] — a valid ordering of all tasks.

Example:

```
tasks = ["fetch_data", "clean_data", "train_model", "evaluate_model"]
dependencies = [("fetch_data", "clean_data"),
               ("clean_data", "train_model"),
               ("train_model", "evaluate_model")]
# Returns: ["fetch_data", "clean_data", "train_model", "evaluate_model"]
```

1.2 Ordering Rules

- Every task in tasks appears exactly once in the returned execution order.
- For every dependency pair (A, B), A appears before B in the output.
- Deterministic output: when multiple tasks are available, the lexicographically largest is selected first.

```
schedule_tasks(["b", "a", "c"], []) # Returns: ["c", "b", "a"]
```

1.3 Error Handling

TypeError	Dependency is not a pair of strings
-----------	-------------------------------------

1.4 Boundary Conditions

Condition	Expected Behavior
Empty tasks and dependencies	Return []
Single task, no dependencies	Return [task]
Multiple tasks, no dependencies	Lexicographic order (largest)
Long dependency chain	Dependency order
Multiple independent chains	Deterministic via lexicographic availability
Diamond-shaped graph	Downstream after all prerequisites

Diamond example:

```
tasks = ["a", "b", "c", "d"] dependencies = [("a", "b"), ("a", "c"), ("b", "d"), ("c", "d")] # Returns: ["a", "b", "c", "d"]
```

1.5 Assumptions

- Standard topological sort is acceptable.
- Task names are case-sensitive.
- Function returns ordering only — does not execute tasks or touch files.
- Expected complexity: $O((N + E) \log N)$ time, $O(N + E)$ space.